LAPORAN PRAKTIKUM PEMROGRAMAN BERORIENTASI ONJEK

PERTEMUAN 4 : CONSTRUCTOR, DESTRUCKTOR, DAN OVERLOADING PADA PHP

1. IDENTITAH MAHASISWA

NAMA : Faizal Dwi Septian

NIM : 2530521061

KELAS : 3 TI C

2. Tujuan Praktikum

- Memahami konsep dasar serta kegunaan dari constructor, destructor, dan overloading pada PHP 8+.
- Bisa membuat dan memanggil objek yang mengimplementasikan konstruktor berparameter serta nilai default.
- Menganalisis mekanisme kerja dan kapan waktu dipanggilnya destructor, serta membedakannya dengan aksi hapus data.
- Mengatur dan membatasi akses properti maupun method dinamis lewat magic methods (__get, __set, __isset, __unset, dan __call).
- Menyelesaikan seluruh soal penugasan latihan mandiri dan menganalisis output programnya.

3. Ringkasan Teorti Tiga Topik

- Constructor (__construct)
  Constructor adalah method khusus yang otomatis berjalan saat objek dibuat dengan kata kunci new. Fungsinya untuk menyiapkan state awal objek agar langsung berisi data yang valid. Di php, terdapat fitur Constructor Property Promotion yang memungkinkan deklarasi visibilitas, tipe data, dan nilai default langsung di dalam parameter constructor tanpa variabel terpisah, membuat kode jauh lebih ringkas.

- Destructor (__destruct)
  Destructor (__destruct()) adalah kebalikan constructor yang otomatis dipanggil saat objek dihapus dari memori RAM atau saat skrip berakhir. Perintah unset($obj) tidak langsung memicu destructor jika objek masih memiliki referensi atau alias lain di memori. Perlu dicatat bahwa destructor hanya

berfungsi untuk pembersihan memori server, bukan untuk menghapus data dari database atau Session (CRUD).

- Overloading dan Magic MethodsOverloading di php berbeda dengan Java/C++ yang mendefinisikan banyak method bernama sama. Di php, overloading adalah mekanisme otomatis saat program mencoba mengakses properti atau method yang tidak tersedia maupun private/protected. Hal ini ditangani oleh Magic Methods (__get, __set, __isset, __unset, dan __call). Fitur ini membuat class lebih dinamis, namun tetap memerlukan validasi atau exception agar struktur kode tetap terkendali.

4. **Source code dan hasil percobaan**
   Modifikasi Construktor Mahasiswa

```
<?php
declare(strict_types=1);

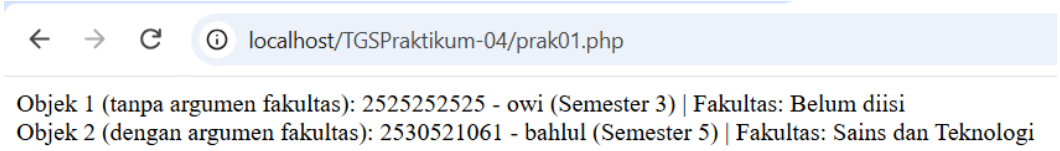
class Mahasiswa {
    public function __construct(
        public string $nim,
        public string $nama,
        public int $semester = 1,
        public string $fakultas = "Belum diisi"
    ) {}

    public function profil(): string {
        return "$this->nim - $this->nama (Semester $this->semester) | Fakultas: $this->fakultas";
    }
}

$m1 = new Mahasiswa('2525252525', 'owi', 3);
$m2 = new Mahasiswa('2530521061', 'bahlul', 5, 'Sains dan Teknologi');

echo "Objek 1 (tanpa argumen fakultas): " . htmlspecialchars($m1->profil()) . "<br>";
echo "Objek 2 (dengan argumen fakultas): " . htmlspecialchars($m2->profil()) . "<br>";
```

output:



5. **Source code dan hasil percobaan**

Class buku

```
<?php
declare(strict_types=1);

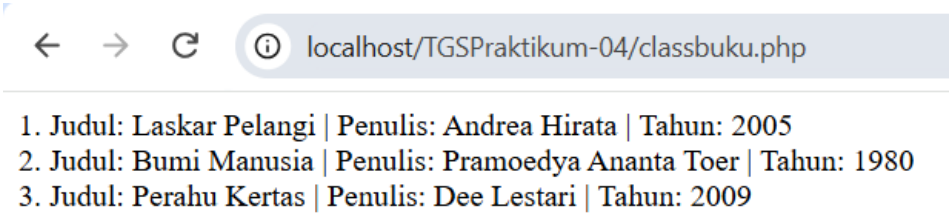
// Latihan 2: Class Buku dengan constructor dan method tampilInfo()
class Buku {
    public function __construct(
        public string $judul,
        public string $penulis,
        public int $tahun
    ) {}

    public function tampilInfo(): string {
        return "Judul: $this->judul | Penulis: $this->penulis | Tahun: $this->tahun";
    }
}

// Inisialisasi 3 objek buku dengan novel Indonesia
$buku1 = new Buku('Laskar Pelangi', 'Andrea Hirata', 2005);
$buku2 = new Buku('Bumi Manusia', 'Pramoedya Ananta Toer', 1980);
$buku3 = new Buku('Perahu Kertas', 'Dee Lestari', 2009);

echo "1. " . htmlspecialchars($buku1->tampilInfo()) . "<br>";
echo "2. " . htmlspecialchars($buku2->tampilInfo()) . "<br>";
echo "3. " . htmlspecialchars($buku3->tampilInfo()) . "<br>";
```

output:



6. **Source code dan hasil percobaan**

Destructor pada class buku

```
<?php
declare(strict_types=1);

// Latihan 3: Destructor pada class Buku + pengaruh alias terhadap unset()
class Buku {
    public function __construct(
        public string $judul,
        public string $penulis,
        public int $tahun
```

```
    ) {
        echo "[Constructor] Buku '$this->judul' dibuat<br>";
    }

    public function tampilInfo(): string {
        return "Judul: $this->judul | Penulis: $this->penulis | Tahun: $this->tahun";
    }

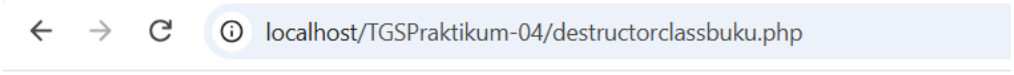
    public function __destruct() {
        echo "[Destructor] Buku '$this->judul' dihancurkan<br>";
    }
}

// Skenario A: unset TANPA alias
echo "<b>Skenario A: unset tanpa alias</b><br>";
$bukuA = new Buku('Laskar Pelangi', 'Andrea Hirata', 2005);
unset($bukuA);
echo "Setelah unset(\$bukuA) -> destructor langsung berjalan (sudah tampil di
atas)<br><br>";

// Skenario B: unset DENGAN alias
echo "<b>Skenario B: unset dengan alias</b><br>";
$bukuB = new Buku('Bumi Manusia', 'Pramoedya Ananta Toer', 1980);
$alias = $bukuB;                // objek kini punya 2 referensi
unset($bukuB);                  // hanya menghapus 1 referensi
echo "Setelah unset(\$bukuB) -> destructor BELUM berjalan, objek masih bisa
diakses lewat alias:<br>";
echo htmlspecialchars($alias->tampilInfo()) . "<br>";
unset($alias);                  // referensi terakhir dihapus
echo "Setelah unset(\$alias) -> destructor berjalan (sudah tampil di atas)<br><br>";

// Skenario C: tanpa unset -> destructor jalan di akhir skrip
echo "<b>Skenario C: tanpa unset</b><br>";
$bukuC = new Buku('Perahu Kertas', 'Dee Lestari', 2009);
echo "Akhir skrip... destructor Perahu Kertas akan berjalan setelah ini.<br>";
```

output:



Skenario A: unset tanpa alias
[Constructor] Buku 'Laskar Pelangi' dibuat
[Destructor] Buku 'Laskar Pelangi' dihancurkan
Setelah unset($bukuA) -> destructor langsung berjalan (sudah tampil di atas)

Skenario B: unset dengan alias
[Constructor] Buku 'Bumi Manusia' dibuat
Setelah unset($bukuB) -> destructor BELUM berjalan, objek masih bisa diakses lewat alias:
Judul: Bumi Manusia | Penulis: Pramoedya Ananta Toer | Tahun: 1980
[Destructor] Buku 'Bumi Manusia' dihancurkan
Setelah unset($alias) -> destructor berjalan (sudah tampil di atas)

Skenario C: tanpa unset
[Constructor] Buku 'Perahu Kertas' dibuat
Akhir skrip... destructor Perahu Kertas akan berjalan setelah ini.
[Destructor] Buku 'Perahu Kertas' dihancurkan

7. **Source code dan hasil percobaan**

Atribut dinamis

```
<?php
declare(strict_types=1);

class MahasiswaDinamis {
   private array $atributTambahan = [];

   public function __construct(public string $nim, public string $nama) {}

   public function __set(string $key, mixed $value): void {
      $diizinkan = ['hobi'];
      if (!in_array($key, $diizinkan, true)) {
         throw new InvalidArgumentException("Atribut '$key' tidak diizinkan");
      }
      $this->atributTambahan[$key] = $value;
      echo "[__set] Atribut '$key' diisi.<br>";
   }

   public function __get(string $key): mixed {
      echo "[__get] Membaca atribut '$key'.<br>";
      if (!array_key_exists($key, $this->atributTambahan)) {
         throw new OutOfBoundsException("Atribut '$key' tidak ditemukan");
      }
      return $this->atributTambahan[$key];
   }

   public function __isset(string $key): bool {
      echo "[__isset] Memeriksa atribut '$key'.<br>";
      return isset($this->atributTambahan[$key]);
   }

   public function __unset(string $key): void {
      echo "[__unset] Menghapus atribut '$key'.<br>";
      unset($this->atributTambahan[$key]);
   }
}

$mhs = new MahasiswaDinamis('231001', 'Alya');

echo "<b>1. Uji __set()</b><br>";
$mhs->hobi = 'Membaca';

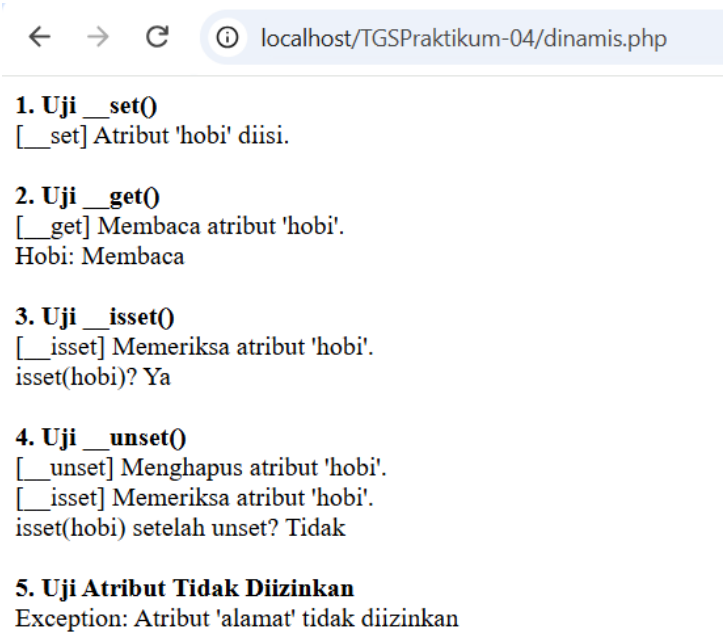
echo "<br><b>2. Uji __get()</b><br>";
echo "Hobi: " . htmlspecialchars($mhs->hobi, ENT_QUOTES, 'UTF-8') . "<br>";
```

```
echo "<br><b>3. Uji __isset()</b><br>";
echo "isset(hobi)? " . (isset($mhs->hobi) ? 'Ya' : 'Tidak') . "<br>";

echo "<br><b>4. Uji __unset()</b><br>";
unset($mhs->hobi);
echo "isset(hobi) setelah unset? " . (isset($mhs->hobi) ? 'Ya' : 'Tidak') . "<br>";

echo "<br><b>5. Uji Atribut Tidak Diizinkan</b><br>";
try {
   $mhs->alamat = 'Sukabumi';
} catch (InvalidArgumentException $e) {
   echo "Exception: " . htmlspecialchars($e->getMessage()) . "<br>";
}
```

Output:



8. **Source code dan hasil percobaan**

Overloading methode

```
<?php
declare(strict_types=1);

class LaporanMahasiswa {
   public function __construct(private string $nama) {}

   public function __call(string $method, array $args): mixed {
      if ($method === 'cetakRingkas' && count($args) === 0) {
         return 'Mahasiswa: ' . $this->nama;
      }
      throw new BadMethodCallException('Method tidak tersedia: ' . $method);
   }
}
```

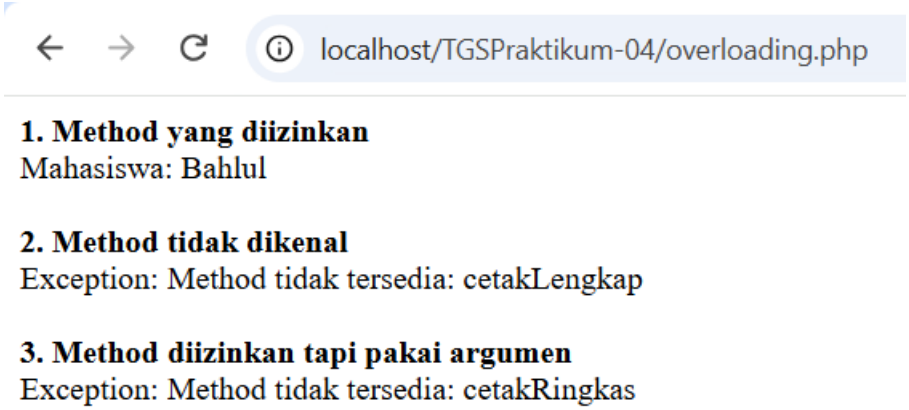
```
$laporan = new LaporanMahasiswa('Bahlul');

echo "<b>1. Method yang diizinkan</b><br>";
echo htmlspecialchars($laporan->cetakRingkas()) . "<br><br>";

echo "<b>2. Method tidak dikenal</b><br>";
try {
    echo $laporan->cetakLengkap();
} catch (BadMethodCallException $e) {
    echo "Exception: " . htmlspecialchars($e->getMessage()) . "<br><br>";
}

echo "<b>3. Method diizinkan tapi pakai argumen</b><br>";
try {
    echo $laporan->cetakRingkas('tambahan');
} catch (BadMethodCallException $e) {
    echo "Exception: " . htmlspecialchars($e->getMessage()) . "<br>";
}
```

Output:



9. **Hasil tabel pengujian**

Kasus Pengujian	Gejala yang Diharapkan	Analisis Singkat	Hasil Pengujian
Parameter constructor kurang	ArgumentCountError	Jumlah argumen wajib yang dikirim pas instansiasi tidak memenuhi parameter constructor	Muncul Fatal Error ArgumentCountError karena argumen mandatory NIM/nama tidak diisi
Tipeparametersalah (strict_types=)	Typer error	Tipe data argumen beda sama kontrak constructor (misal angka/int diisi huruf)	Muncul TypeError karena tipe data gak cocok
Menulis atribut terlarang via __set()	InvalidArgumentException	Nekat ngisi properti di luar daftar whitelist yang dibolehkan sistem.	Muncul exception bertuliskan "Atribut tidak diizinkan".
Memanggil method tidak dikenal via __call()	BadMethodCallException	Manggil method yang gak pernah dideklarasikan di class.	Muncul exception "Method tidak tersedia".
Unset objek yang punya variabel alias	Destructor belum berjalan	Variabel objek di-unset, tapi objeknya masih dipegang sama variabel alias.	Destructor gak langsung jalan, baru aktif pas alias terakhirnya ikut dihapus.

10. Analisis urutan Constructor-Methode-Destructor

1. Inisialisasi (Constructor): Saat new dipanggil, PHP mengalokasikan memori dan langsung menjalankan __construct() untuk mengisi properti awal.
2. Operasional (Method): Objek aktif di memori dan bisa memanggil method publik (seperti tampilInfo()) lewat operator ->.
3. Terminasi (Destructor): Saat objek kehilangan seluruh referensinya, PHP otomatis memanggil __destruct() untuk membersihkan memori.

Perintah unset() cuma menghapus nama variabel (referensi) yang menunjuk ke objek, bukan langsung menghancurkan objeknya kalau masih ada pegangan lain. Contohnya pada Skenario B Latihan 3, objek 'Bumi Manusia' punya dua referensi ($bukuB dan $alias). Pas $bukuB di-unset, destructor belum jalan karena objek masih dipegang oleh $alias. Destructor baru benar-benar tereksekusi pas $alias ikut di-unset.

Destructor sering disangka sama dengan fungsi hapus data (CRUD). Padahal, destructor murni 100% mengurus pembersihan memori RAM server saat objek tidak terpakai atau skrip selesai. Sebaliknya, method hapus() pada CRUD bertugas menghapus data bisnis secara permanen dari media penyimpanan seperti Session, Array, atau Database.

11. Analisis pemicu magic methode

Magic methode pada PHP adalah methode khusus dengan awalan dua garis bawah (__) yang bakal jalan secara otomatis sesuai pemicu (trigger) tertentu pas kita berinteraksi sama property atau methode yang gak terakses (privat,protected) atau gak ada sama sekali.

- __set($nama, $nilai): Terpanggil otomatis pas kita mencoba mangisi nilai ke property yang gak dideklarasikan di class. $mhs->hobi = 'Membaca' memicu __set(). Karena 'hobi' ada di daftar whitelist, nilainya disimpan ke array internal.
- __get($nama): Terpanggil otomatis pas kita mencoba membaca nilai dari properti dinamis. Di Latihan 4, echo $mhs->hobi memicu __get() buat mengambil nilai dari array internal.
- __isset($nama): Terpanggil pas fungsi isset() atau empty() dipakai ke properti dinamis, kaya isset($mhs->hobi).
- __unset($nama): Terpanggil pas fungsi unset() dipakai ke properti dinamis, kaya unset($mhs->hobi) buat menghapus elemen di array internal.
- __call($nama, $argumen): Terpanggil pas kita memanggil *method instance* yang gak dideklarasikan di *class*. Di Latihan 5, pemanggilan $laporan->cetakRingkas() ditangkap sama __call().

Penggunaan magic methods bikin class kita fleksibel banget karena bisa nampung atribut atau method ekstra tanpa perlu ngetik variabel satu-satu. Tapi risikonya, kalau kita gak kasih validasi ketat seperti Exception, salah ketik nama variabel typo gak bakal dianggap error sama PHP, yang ada malah bug nya makin susah dicari.

12. Evaluasi dan refleksi

Apa perbedaan constructor dengan method biasa?

Constructor (__construct) adalah method khusus yang berjalan secara otomatis begitu objek dibuat lewat perintah new. Sebaliknya, method biasa tidak bisa jalan sendiri secara otomatis dan wajib dipanggil secara manual menggunakan operator panah (->).

Mengapa nilai awal objek sebaiknya valid sejak dibuat?

Agar objek langsung berada dalam kondisi (state) yang konsisten dan siap digunakan tanpa memicu error. Dengan memastikan data awal seperti NIM dan nama sudah valid sejak objek dibentuk, kontrak pembuatan objek telah terpenuhi sebelum program memanipulasi datanya.

Apakah unset($obj) selalu langsung menjalankan destructor? Jelaskan.

Tidak selalu. Perintah unset($obj) hanya bertugas menghapus satu nama variabel referensi yang menunjuk ke objek. Jika objek tersebut masih memiliki referensi atau variabel alias lain yang aktif di memori, destructor (__destruct) tidak akan berjalan sampai seluruh referensinya benar-benar habis.

Mengapa destructor tidak sama dengan method hapus() dalam CRUD?

Destructor murni berfungsi untuk membersihkan objek dari memori RAM server saat skrip selesai atau referensinya habis, bukan untuk menghapus data bisnis. Sementara itu, method hapus() pada CRUD bertugas menghapus data atau record secara permanen dari media penyimpanan seperti Session, Array, atau database.

Apa perbedaan overloading PHP dengan method overloading di Java/C++?

Di Java atau C++, overloading berarti membuat beberapa method bernama sama dengan parameter yang berbeda. Sedangkan di PHP, overloading adalah mekanisme otomatis yang terpicu melalui magic methods (__get, __set, __call, dll.) ketika program mencoba mengakses properti atau method yang tidak tersedia maupun tidak dapat diakses.

Kapan penggunaan __get/__set bermanfaat, dan apa risikonya?

- __get/__set sangat bermanfaat ketika kita ingin menangani atribut tambahan yang sifatnya dinamis dan opsional tanpa harus mendefinisikan banyak properti di dalam class.

- Risikonya adalah dapat melemahkan kejelasan struktur class dan menyembunyikan bug. Jika programmer salah mengetik nama properti (typo), PHP tidak akan memunculkan error fatal, melainkan diam-diam memperlakukannya sebagai properti dinamis

13. Kesimpulan

Secara keseluruhan, constructor(__construct)sangat membantu dalam menyiapkan data objek agar langsung valid sejak pertama kali dibuat pakai (new), dipercepat juga dengan fitur Constructor Property Promotion di PHP, Sementara itu destructor(__destruct)berfungsi murni untuk membersihkan memori RAM server saat referensi objek habis, sehingga jelas berbeda fungsi dengan operasi hapus data. Selain itu perintah (unset()) tidak langsung memicu destructor selama objek masih memegang wariabel alias memori

Di sisi lain, Overloading lewat magic methode (__set, __get, __isset, __unset, dan __call) memberikan fleksibilitas tinggi untuk menangani property atauh methode dinamis. Meskipun sangat membantu, penggunaannya wajib dibarengi validasi dan Exception yang ketat agar kode tetap aman serta terhindar dari bug tersembunyi akibat salah ketik (typo)

14. Link github

https://github.com/zeyn-lgtm/PBO_P4